

시간복잡도 - 알고리즘이 얼마나 빠른지 재는 자

왜 배워야 할까?

같은 문제를 풀어도 방법에 따라 걸리는 시간이 다르다. KO는 제한 시간이 1~2초다. 느린 방법은 시간 초과로 0점이다. 시간복잡도는 "입력 n 이 커질 때 연산 횟수가 어떻게 늘어나는가"를 보는 자다.

비유: 30명 출석 부르기 - 선생님이 맨 앞자리만 본다 → 1번 보면 끝. $O(1)$ - 30명 이름을 차례로 부른다 → 30번. $O(n)$ - 30명끼리 서로 인사한다 → $30 \times 30 = 900$ 번. $O(n^2)$

1. Big-O는 최악을 본다

알고리즘은 빠를 때도, 느릴 때도 있다. 채점은 가장 느린 경우로 한다. 그래서 Big-O는 최악의 경우 연산 횟수의 증가 형태만 남긴다.

$O(1)$: n 이 커져도 1번이면 끝
 $O(\log n)$: n 이 2배가 돼도 1번만 더 하면 된다 (반으로 나누기)
 $O(n)$: n 만큼 비례해서 늘어난다
 $O(n \log n)$: n 번 하는데 한 번 할 때마다 $\log n$ 만큼 더 든다 (정렬)
 $O(n^2)$: 이중 반복문. $n=1000$ 이면 100만 번
 $O(2^n)$: 모든 부분집합을 다 본다. $n=30$ 이면 10억 번
 $O(n!)$: 모든 순서를 다 본다. $n=10$ 이면 360만 번

flowchart LR

```

A[입력 n] --> B{어떤 알고리즘?}
B -->|한 번만 접근| C[01\ n예: arr[0]]
B -->|반씩 나누기| D[0 log n\ n예: 이진탐색]
B -->|한 번 훑기| E[0 n\ n예: 전체 합 구하기]
B -->|훑고 나누기| F[0 n log n\ n예: 병합정렬]
B -->|이중 훑기| G[0 n^2\ n예: 버블정렬]
B -->|모든 경우| H[0 2^n / n!\ n예: 모든 부분집합]
  
```

증가 속도 그림으로 보기

xychart-beta

```
title "n이 커질 때 연산 횟수 증가"
x-axis [10, 100, 1000, 10000]
y-axis "연산 횟수" 0 --> 100000000
line [1, 1, 1, 1]
line [3, 6, 9, 13]
line [10, 100, 1000, 10000]
line [33, 664, 9965, 132877]
line [100, 10000, 1000000, 100000000]
```

읽는 법: $n=1000$ 에서 $O(n^2)$ 은 100만, $O(n \log n)$ 은 1만보다 작다. K0I에서 $n=100,000$ 이면 $O(n^2)$ 은 절대 통과 못 한다.

2. 자주 나오는 복잡도 판별법

코드 형태	복잡도	판단 이유
<code>arr[0] , a+b</code>	$O(1)$	n과 상관없이 항상 1번
<code>for(i=0;i<n;i++)</code> 한 개	$O(n)$	n번 돈다
for 안의 for	$O(n^2)$	$n \times n$
<code>while(n>1) n/=2</code>	$O(\log n)$	반씩 줄인다
<code>sort()</code>	$O(n \log n)$	C++ 정렬은 이 속도
부분집합 전부 생성	$O(2^n)$	각 원소마다 넣는다/안 넣는다 2가지

핵심 규칙: 가장 크게 늘어나는 항만 남긴다. $n^2 + n + 1 \rightarrow O(n^2)$. n이 10만이면 n은 n^2 에 비해 의미 없다.

Ω, Θ는 언제 쓰나?

- 0: 최악. "아무리 느려도 이보다는 빠르다" - 채점 기준
- Ω: 최선. "아무리 빨라도 이보다는 느리다"
- Θ: 최선과 최악이 같을 때. "정확히 이 정도"

KOI에서는 0만 제대로 알면 된다.

3. KOI 시간 제한으로 거꾸로 계산하기

1초에 C++는 약 1억 번 연산한다.

n의 크기	통과 가능한 복잡도	KOI에서 자주 보이는 n
$n \leq 10$	$O(n!)$ 가능	순열 전부 보기
$n \leq 20$	$O(2^n)$ 가능	부분집합
$n \leq 1,000$	$O(n^2)$ 가능	이중 반복문
$n \leq 100,000$	$O(n \log n)$ 까지	정렬 필요
$n \leq 1,000,000$	$O(n)$ 까지	한 번 훑기

flowchart TD

Q[문제에서 n을 본다] --> R{n은 몇?}

R --> | $n \leq 20$ | S[2^n 도 가능\브루트포스·비트마스킹 고려]

R --> | $n \leq 2000$ | T[n^2 가능\이중 반복문·DP $O(n^2)$]

R --> | $n \geq 100000$ | U[$n \log n$ 이하만 가능\정렬·이분탐색· $O(n)$ 설계]

U --> V[$O(n^2)$ 쓰면 시간초과 확정]

예: $n=200,000$ 인데 이중 반복문을 쓰면 400억 번. 1초 안에 절대 못 끝난다. → 정렬 후 한 번 훑는 방법으로 바뀌어야 한다.

4. 직접 손으로 계산해 보기

문제 1. 다음 코드의 시간복잡도는?

```
int sum = 0;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        sum += i + j;
    }
}
```

▶ 풀이 보기

문제 2. $n=1,000,000$, 1초 제한일 때 $O(n^2)$ 알고리즘을 쓰면?

▶ 풀이 보기

5. KOI에서는 이렇게 나온다

- 입력 제한을 먼저 본다. $n \leq 500,000$ 이라고 쓰여 있으면 $O(n^2)$ 풀이는 처음부터 제외한다.
- "정렬"이 필요해 보이면 $O(n \log n)$ 을 예산에 넣는다.
- 시간복잡도를 모르면, 맞게 풀어도 시간초과로 0점이 된다.

다음 장인 정렬에서 $O(n \log n)$ 이 왜 나오는지 그림으로 확인한다.